

GPU Sharing and Coordinated Scheduling for Collaborative LLM Inference

Qingyang Guo Weizhen Zhou Yihao Huang

qg26@fsu.edu wz26c@fsu.edu yh26e@fsu.edu

1 Introduction & Motivation

Our project aims to focus on the scenario of running LLMs on multiple distributed home devices. The single device in this scenario cannot run LLM independently. The system like distributed-llama [1] connects home devices into a cluster to accelerate LLM inference. However, it only serves one request at a time (non-preemptive request-level FCFS), fixes the parallel strategy as Tensor Parallelism (TP), and does not consider dynamic change of strategy with different workload pattern. Which, we assume, could result in several problems: (i) serial scheduling could lead to head-of-line blocking, (ii) TP's synchronous communication overhead could be heavy and limit the advantage of multi-gpu, (iii) as the communication cost is fixed in TP and independent of how many requests are being processed, serial scheduling would be inefficient, (iv) distributed llama fix the parallelism strategy as TP, but when there are many enough active requests, using pipeline parallelism (PP) could be more efficient.

In this project, we plan to

- Conduct wholesome measurement to verify the above assumptions.
- Implement a two-layer adaptive controller for dynamic selection of scheduling strategy and parallelism strategy.

2 The State of the Art

LLM inference scheduling. Sarathi-Serve [2] introduces iteration-level scheduling for datacenter GPUs: it splits the prefill of new requests into chunks and co-schedules them with the decode steps of in-flight requests, so multiple requests share the fixed cost of each iteration without stalling one another. This confirms our observation that batching requests into the same iteration is nearly free throughput. However, such systems assume fast interconnects and treat the parallel strategy as a deployment-time constant. In a home cluster, tensor-parallel synchronization dominates, and concurrency directly changes whether tensor or pipeline parallelism is preferable—a coupling the scheduler never considers.

Distributed inference on home devices. `prima.cpp` [3] targets the same setting as distributed-llama but adopts piped-ring pipeline parallelism with weight prefetching, assigning layers to heterogeneous devices by their compute, memory, and network capacity. Its results show that parallel layout strongly affects home-cluster performance and that pipeline parallelism can beat tensor parallelism in some configurations, consistent with our observation. Yet, like distributed-llama, it fixes the layout at deployment time and serves one request at a time; it can neither schedule among concurrent requests nor switch layouts as concurrency changes.

3 Methodology

3.1 Measurement

For the scheduling strategy, workload scenarios, measurement metrics and expected evaluate success performance, see details in Section 4.

3.2 Adaptive controller

Since we are not considering a heterogeneous scenario, there's no need to use DP here. And we tentatively decide to use a classic system: cost model driven configuration selection and dual-time-scale control. Including a shared performance model, a per-iteration inner scheduler, and an outer layout selector.

Performance model. Do a one-time profiling at the beginning to measure parameters. And predicts the latency of any parallel layout under any scheduling method.

Inner scheduler. Runs every iterations and uses the model to decide the scheduling policy - serial, round-robin or concurrent, and which requests join the next batch, so fixed costs are shared while short requests are not starved.

Outer layout selector. Runs only when load changes noticeably; compares different parallel strategies (PP, TP, hybrid). Considering switch cost, and lazy-switching so to avoid oscillation led by noises.

Implementation details will be provided in the final report.

4 Experiments

4.1 Metrics

We will measure the performance of each schedule strategy in TP and PP by the measuring Time to First Token (TTFT), Time per Output Token (TPOT), Time to Last Token (TTLT), P99 latency, Per-step time, Throughput, Slowdown, Fairness, and Layout Switching Cost.

4.2 Expected Evaluation of Success

For the assumptions in Section 1, we expect such results: In this project, we plan to

- Serial scheduling could lead to large latency for short requests.
- Fixing a request, as the number of GPUs increases(2,4,8), the latency decreases and then increases.
- Fixed GPU numbers and strategy. Increase batch number. It is expected to see that the time for each step only increases slowly, and the throughput grows nearly linearly, indicating that multiple requests share the same fixed communication cost.
- Compare TP and PP on same set of GPUs, with request number increasing, TPOT of PP and TP would have a cross.

For adaptive controller the expected results:

- Comparing to original distributed-llama(TP+serial). Our method has overall better performance on metrics like TTFT, TPOT, P99, fairness, etc. across different workloads described in Section 4.3.

4.3 Repeatable Workloads

We will evaluate all configurations using short prompts sampled from ShareGPT [4] and long prompts sampled from LongBench [5]. Furthermore, all configurations will be run on Llama 3.1 8B Instruct [6] and Llama 3.3 70B Instruct [7], both are 4-bit (Q40) quantized weight provided by distributed llama and will run on its Vulkan backend and same random seed. Three scenarios will be considered: two short requests arriving together; a long-input request arriving while another task is decoding; and a short request arriving while another task is generating a long output. Inputs, arrival schedules, and generation settings will be matched across policies, with solo runs providing references for per-task slowdown.

4.4 Hardware Access

All experiments of this project will be conducted on the HiPerGator cluster. We will perform experiments on “hpg-turin” partition. We will request 2, 4, or 8 NVIDIA L4 GPUs with 24 GB VRAM each for experiment. Since there are 3 NVIDIA L4 GPUs on each node, the experiment will be performed across nodes and will request 1 NVIDIA L4 GPU on each node to simulate a home device.

5 Show Stoppers and Plan B

One obvious challenge is exchange of TP and PP requires reallocate weights and KV cache on device clusters. This might result in heavy overhead, surpass any benefits. Furthermore, distributed-llama does not implement PP. We may stop implementing PP if we find the workload is too much to completing in 10 weeks. In addition, if we cannot implement adaptive controller, we will abort the implementation and record the failure then only complete the measurement part.

6 Timeline

Period	Milestone / Deliverable
Week 1 – 2	Run llama on NVIDIA L4
Week 3 – 4	Measure serial blocking & GPU scaling; implement batching & PP
Week 5 – 6	Implement round-robin; measure batch scaling
Week 7 – 8	TP vs. PP comparison; implement adaptive controller
Week 9 – 10	Analysis & Report

7 Division of Labor Across Team Members

All team members contributed equally to this project.

References

- [1] B. Tadych, “Distributed Llama,” GitHub repository, commit 59af889, 2026. [Online]. Available: <https://github.com/b4rtaz/distributed-llama>
- [2] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, “Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 117–134.
- [3] Z. Li, T. Li, W. Feng, R. Xiao, J. She, H. Huang, M. Guizani, H. Yu, Q. Ho, and W. Xiang, “Prima.cpp: Fast 30-70B LLM inference on heterogeneous and low-resource home clusters,” 2025, arXiv:2504.08791.
- [4] ShareGPT Team, “ShareGPT conversations (ShareGPT_V3_unfiltered_cleaned_split),” Hugging Face dataset mirror, 2023. [Online]. Available: https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered
- [5] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, “LongBench: A bilingual, multitask benchmark for long context understanding,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 3119–3137.
- [6] B. Tadych, “Llama 3.1 8B Instruct, 4-bit (Q40) weights in Distributed Llama format,” Hugging Face model repository, 2024. [Online]. Available: https://huggingface.co/b4rtaz/Llama-3_1-8B-Q40-Instruct-Distributed-Llama
- [7] B. Tadych, “Llama 3.3 70B Instruct, 4-bit (Q40) weights in Distributed Llama format,” Hugging Face model repository, 2024. [Online]. Available: https://huggingface.co/b4rtaz/Llama-3_3-70B-Q40-Instruct-Distributed-Llama